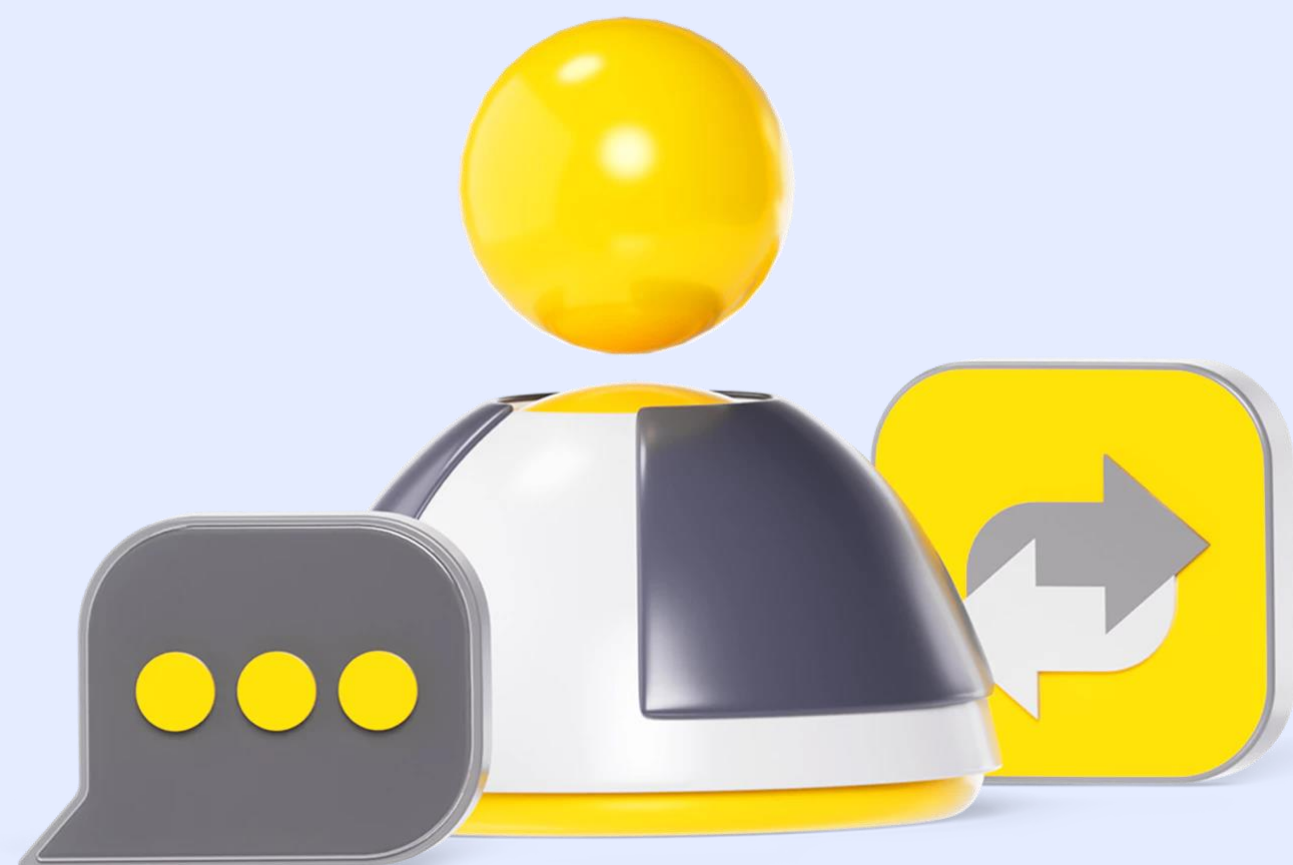


Node.js без Nest

Практическое руководство по чистой архитектуре

Привет 🖐️



Игорь Антонов

Team Lead в Т-Бизнес



Помогаю делать конференции [Podlodka Java Crew](#), [Podlodka TechLead Crew](#), [Podlodka React Crew](#)



Читаю лекции в [HTML Academy](#), ИТМО



Веду канал на YouTube и Telegram — <https://youtube.com/@antonovjs> и <https://t.me/antonovjs>

Про опыт



20 лет в IT



Четыре компании



Разный стэк

О чём пойдёт речь



1

Архитектура ПО

2

Как легко сделать плохо

3

Чистая архитектура

4

Эпилог

«Как имплементировать чистую
архитектуру?»



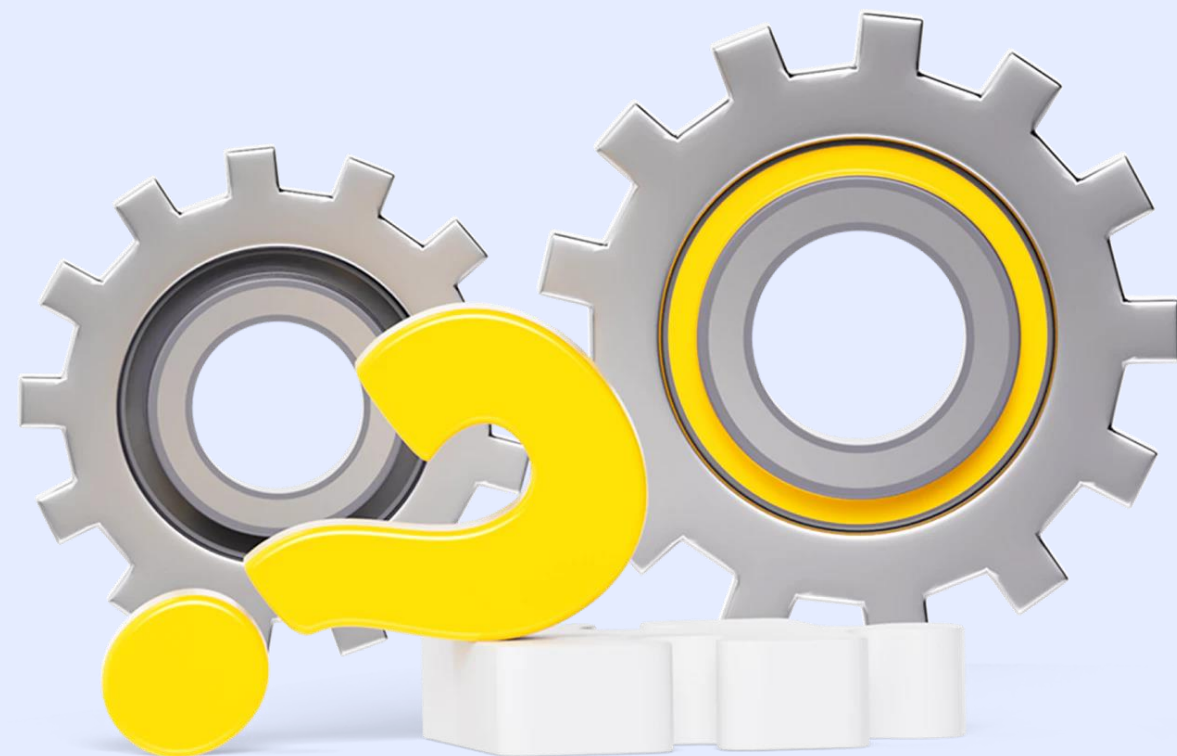
Введение в архитектуру

Жизнь...



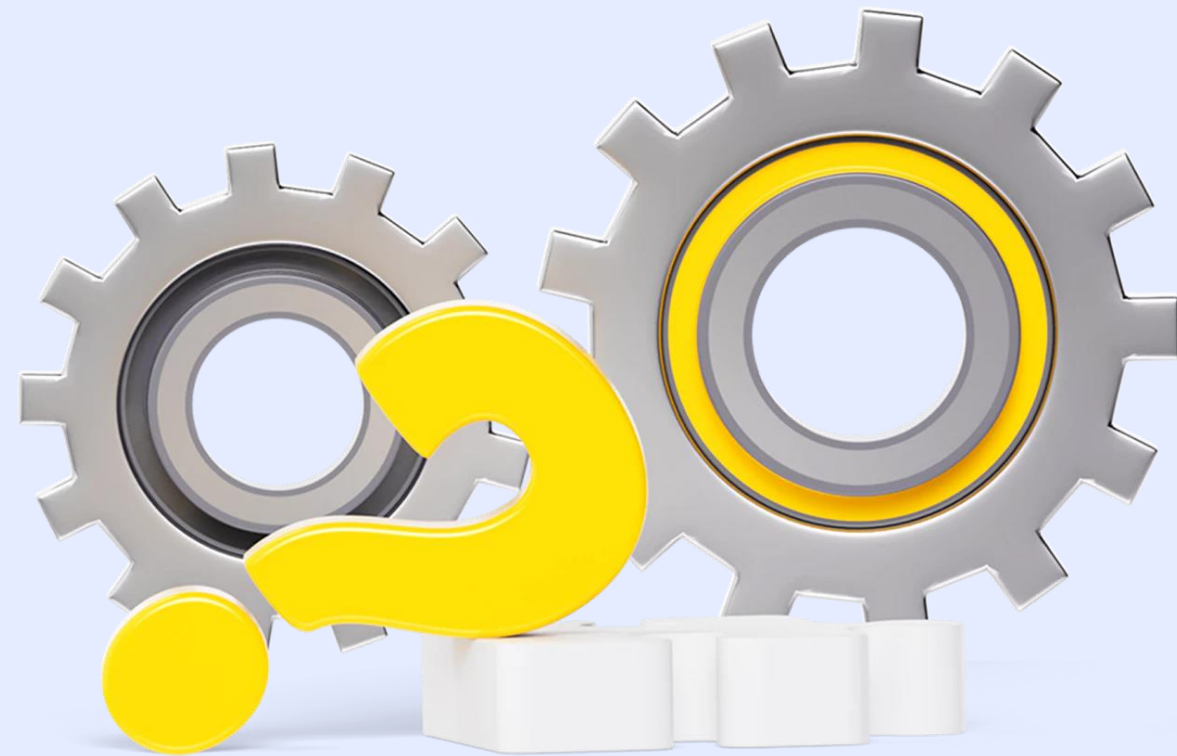
- ➡ Программировать, чтобы оно «заработало»
- ➡ Программировать, чтобы оно работало «правильно»
- ➡ Программировать, чтобы не испытывать боль с поддержкой
- ➡ Перестать программировать 😊

Архитектура



- ➔ Это файловая структура? Чтобы разложить все файлы по полочкам.
- ➔ Это «правильный» фреймворк?
- ➔ Микросервисы vs Монолиты?
- ➔ Доступность? Надёжность?
Тестируемость? Отказоустойчивость?

Архитектура



~150 определений



Наиболее важные характеристики
системы

*«Архитектура — самые важные решения,
которые принимаются перед началом
разработки»*

Верхнеуровневый
дизайн

Архитектура
приложения

Производительность

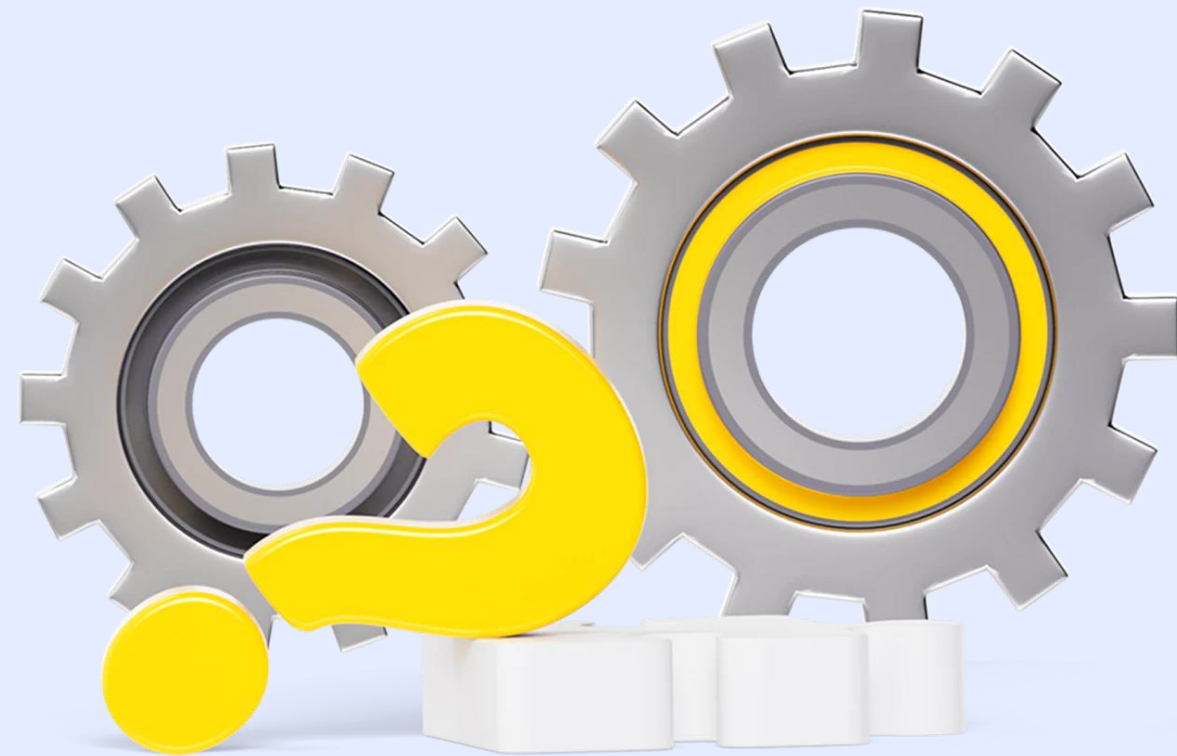
Безопасность

Доступность

Ког



Разработчик и архитектура



Простая поддержка



Отсутствие хрупкости



Расширяемость



Тестируемость





Разные подходы к архитектуре

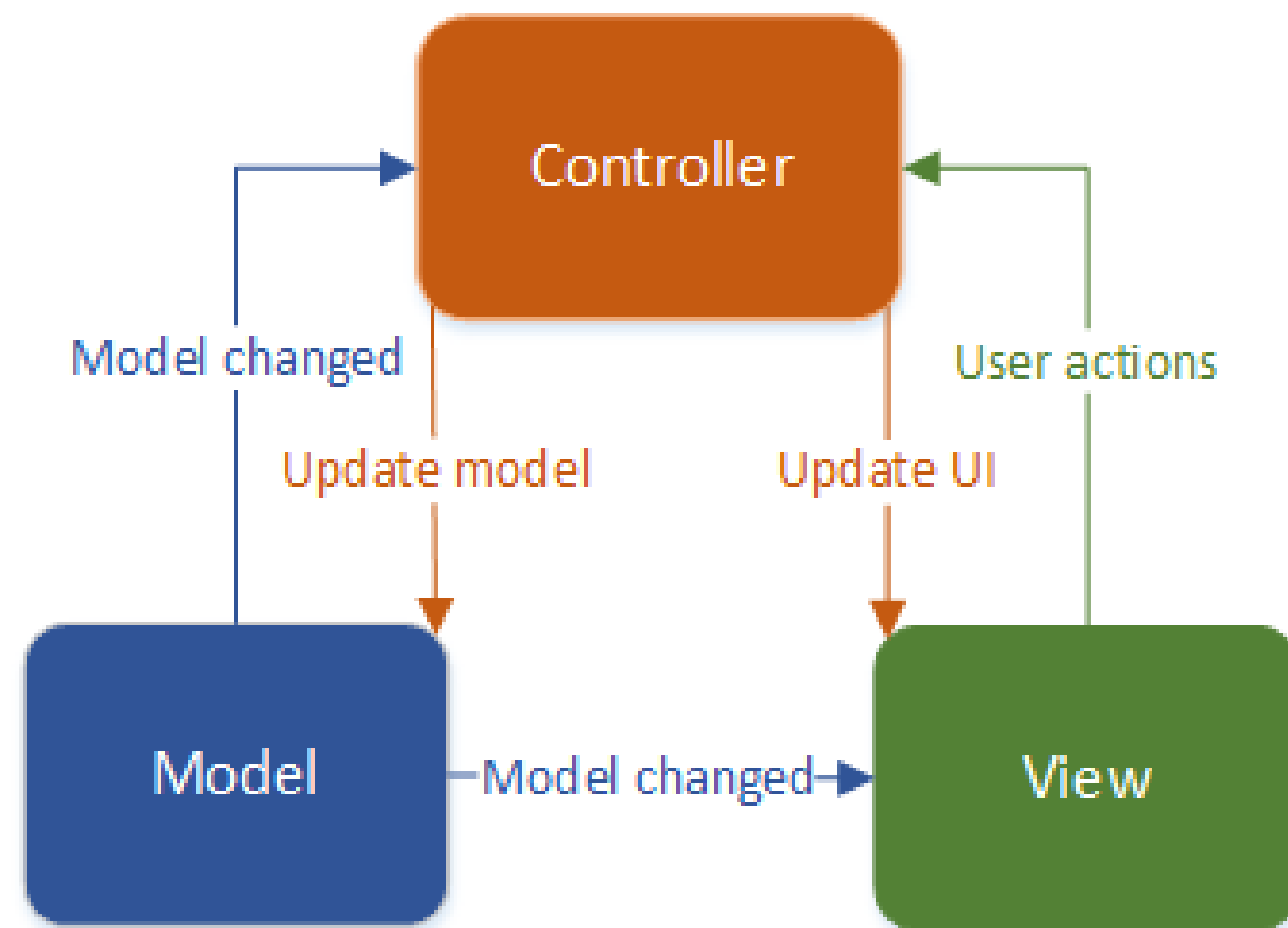


Архитектура

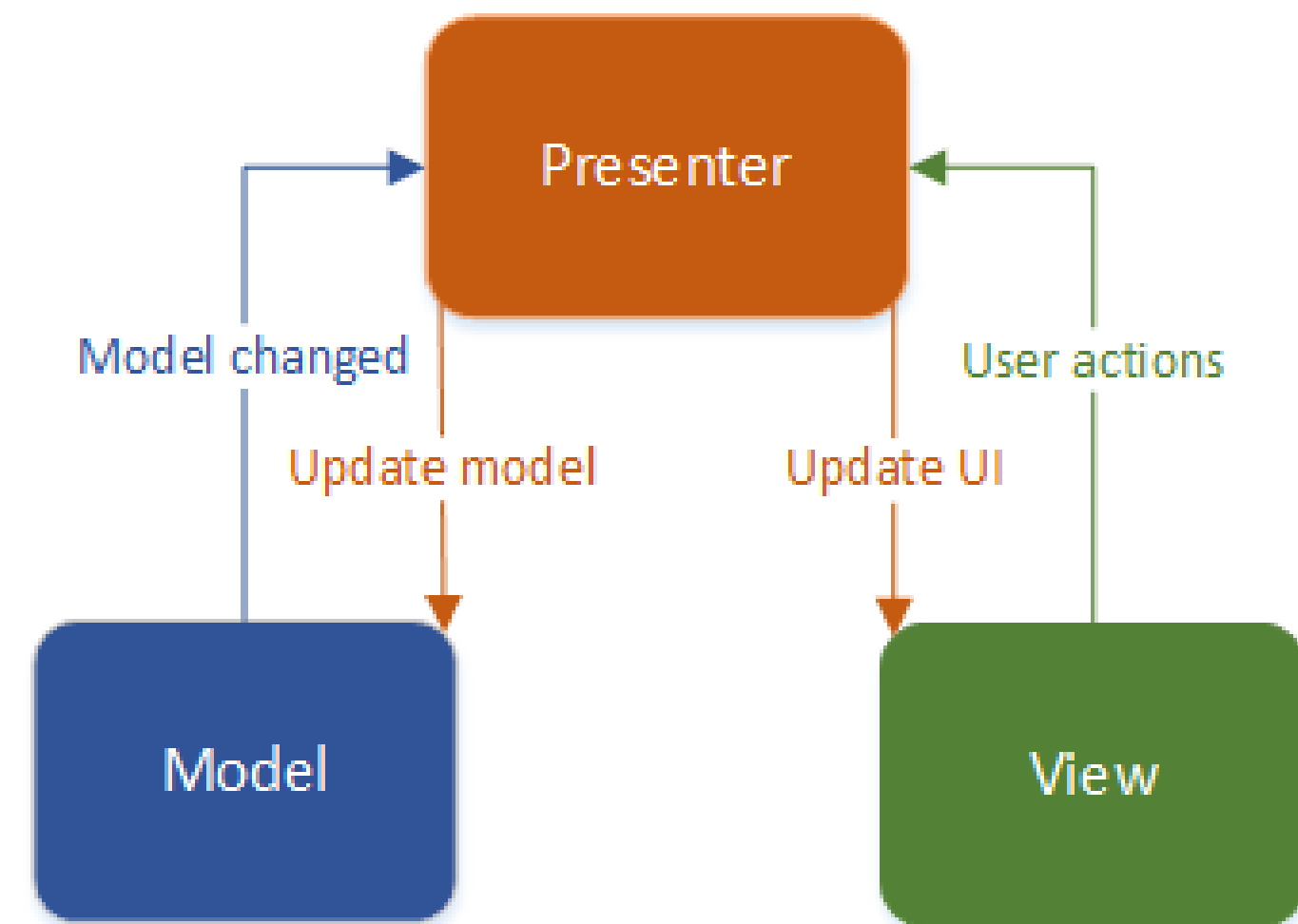


- ➔ Разделение ответственности
- ➔ Модули (компоненты)
- ➔ Публичные контракты
- ➔ Связи

MVC



MVP



Пример «простой» архитектуры



Чистая архитектура

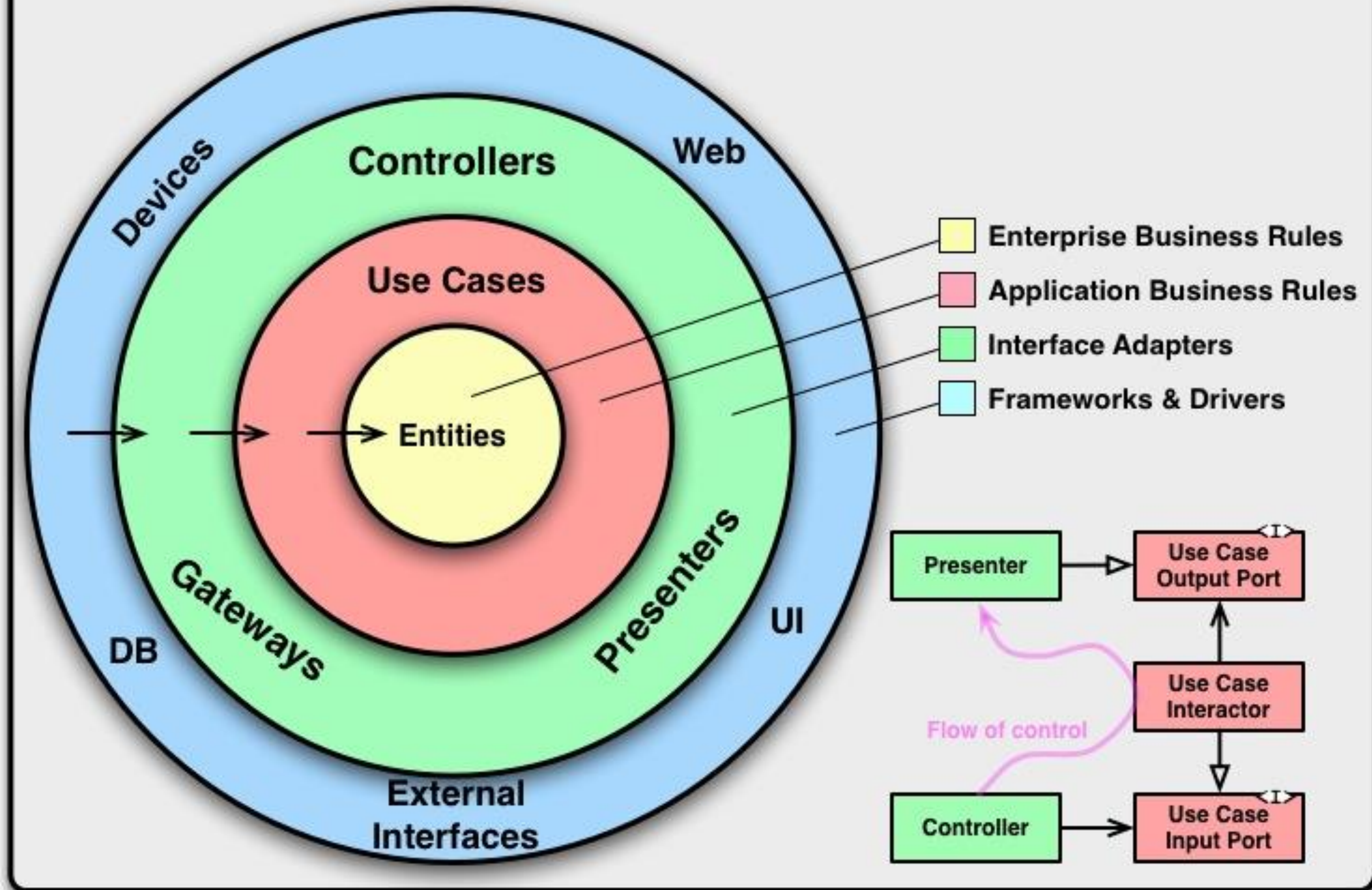


Роберт Мартин



*«Чистая архитектура — архитектурный
подход, методология по организации
кода»*

The Clean Architecture



Чистая архитектура



- ➔ **Цель архитектуры** — управлять сложностью. Чем сложнее система, тем важнее иметь чистую и понятную архитектуру.
- ➔ Архитектура **не должна зависеть** от фреймворков, баз данных, UI и других деталей реализации
- ➔ Хорошую архитектуру **легко изменять и поддерживать.**
- ➔ Архитектура должна поддерживать принципы инверсии зависимостей. Зависимости направлены от абстракций к реализациям, а не наоборот.
- ➔ Чистая архитектура **не должна зависеть** от языка программирования

Чего хотелось лично мне



Стандартизации



Переиспользовать подходы
между проектами



Упростить поддержку



Модульность, гибкость



Слой в чистой архитектуре

Entities



Сущности — самые важные компоненты приложения (внутренний слой). Объекты предметной области, бизнес-процессов. Бизнес-правила.



Сущности **не зависят** от других компонентов приложения



Сущности должны легко тестироваться

```
1 class Order {
2   private status: OrderStatus;
3
4   confirm(): void {
5     if (this.status !== OrderStatus.PENDING) {
6       throw new BusinessRuleError('Only pending orders can be confirmed');
7     }
8     this.status = OrderStatus.CONFIRMED;
9   }
10 }
```

Use Cases



Оркестратор для потока данных между сущностями



Слой определяет **действия**, которые могут быть выполнены с помощью сущностей



Бизнес-правила, не зависит от UI, БД или других сервисов

```
1 class CreateOrderUseCase {
2   async execute(request: CreateOrderRequest): Promise<Order> {
3     const customer = await this.customerRepo.findById(request.customerId);
4     const products = await this.getProducts(request.items);
5
6     const order = Order.create(customer, products);
7     await this.orderRepo.save(order);
8
9     return order;
10  }
11 }
12
```

Interface Adapters



Преобразуют данные из формата сценариев и сущностей для использования с внешними системами.



Доступ к данным, контроллеры, внешние сервисы, фоновые задания, репозитории и так далее.

```
1 class OrderController {
2   async createOrder(req: Request, res: Response): Promise<void> {
3     const result = await this.createOrderUseCase.execute(req.body);
4     res.json(this.presenter.present(result));
5   }
6 }
7
8 class DatabaseOrderRepository implements OrderRepository {
9   async save(order: Order): Promise<void> {
10    // Сохранение в БД
11  }
12 }
13
```

Frameworks и Drivers



Внешние инструменты и библиотеки для выполнения специфичных задач.

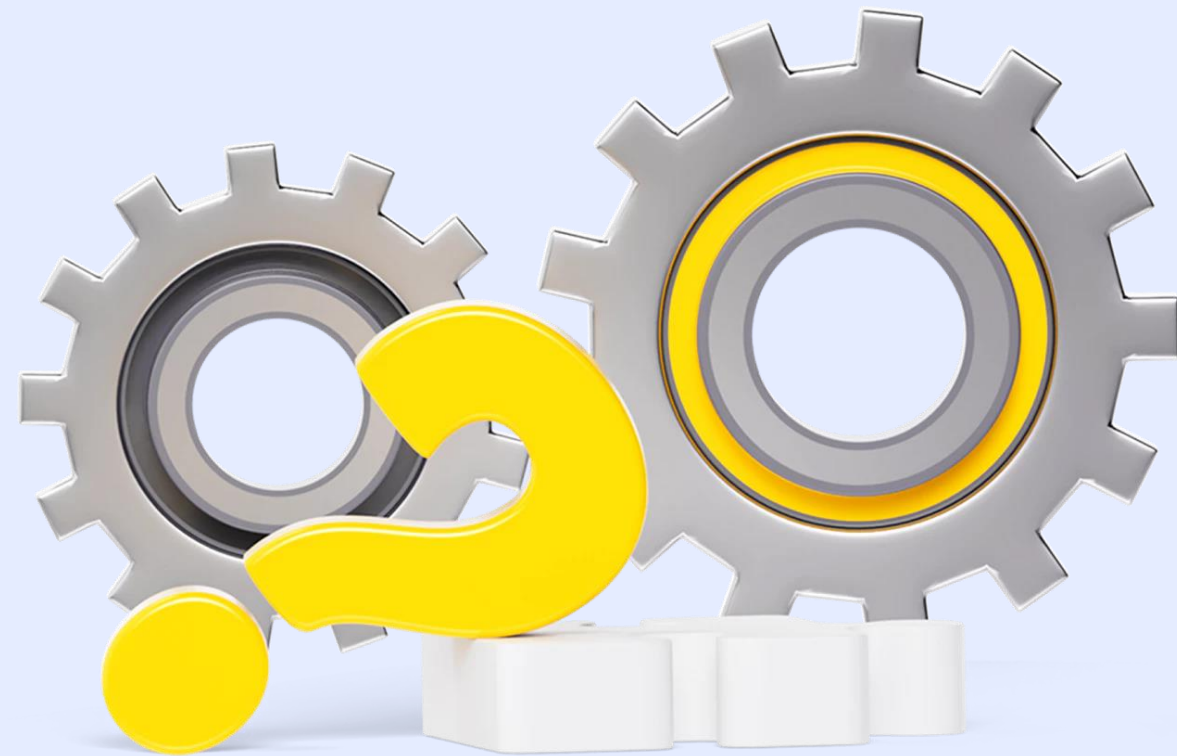


Базы данных, фреймворки и так далее.



```
1  
2  app.post('/orders', orderController.createOrder);  
3
```

Преимущества чистой архитектуры



Тестируемость



Гибкость. Легко заменить внешний инструмент



Переиспользование. Бизнес-логика не привязана к фреймворкам



Командная работа. Разделение ответственности



DDD (Domain-Driven Design)

DDD



Методология проектирования ПО.
Бизнес-логика в центре внимания



Сложность бизнес-логики
должна быть изолирована



Код отражает реальные бизнес-
процессы

Entity



Объект с уникальной идентичностью



Богатые vs анемичные модели

Value Object



Объект без идентичности, определяется только своими атрибутами



Два объекта с одинаковыми значениями считаются равными



Неизменяемость

Aggregate



Группа связанных Entity и Value Objects, который снаружи рассматривается как единое целое. Все сущности доступны через Aggregate Root



Пример: заказ в интернет-магазине



Архитектура и реальный мир

Материалы



Примеры к докладу —

https://github.com/AntonovIgor/clean-architecture-in-nodejs_2025



Пример nTier архитектуры —

<https://github.com/AntonovIgor/it-conf30>



Блог в Telegram — <https://t.me/antonovjs>



Вопросы? https://t.me/antonov_i

Выводы

Не универсальное
решение



Явное разделение
ответственности



Проще тестировать



Гибкая разработка



Проще мигрировать





Спасибо!